# Day 1 Recap

## What We Learned Today

KavinSchool

# Day 1: What We Covered

## Session 1: Foundations

- AI → ML → DL → Gen AI evolution
- How LLMs work: Transformers, tokens, embeddings, attention
- The model landscape: open vs closed, capability vs cost

## Session 2: Working with the API

- Provider-agnostic setup with `litellm`
- Messages, roles, temperature, max_tokens
- Multi-turn conversations and stateless nature

# Day 1: What We Covered

## Session 3: Capabilities & Limitations

- What LLMs excel at vs where they fail
- Hallucination mitigation strategies
- Input preprocessing, chunking, PII handling
- Token counting and cost estimation

## Session 4: Hands-on Labs

- Code Explainer, Bug Triager, User Story Generator

# Day 2 Preview: Prompt Engineering Deep Dive

Tomorrow we go deeper into the art and science of prompting:

- **Prompt Engineering Frameworks:** 4 Ss, RTF, GCES, RISEN
- **Advanced Techniques:** Chain-of-Thought, Few-Shot, Tree of Thoughts
- **Output Control:** JSON mode, structured outputs, guardrails
- **Prompt Optimization:** iterating, testing, benchmarking
- **Labs:** Build a Code Reviewer, a Meeting Summarizer, a Prompt Library

> **Homework (optional):** Experiment with different system messages in your Lab 1.2 CLI Q&A. Try making the chatbot a pirate, a professor, or a code reviewer.

# Quick Knowledge Check

1. What does **LLM** stand for and how does it generate text?
2. What are the **three roles** in the messages array?
3. If `finish_reason` is `"length"`, what happened?
4. Name **two things** LLMs are bad at.
5. Why should you **count tokens** before sending a prompt?
6. What is the purpose of the **system message**?

# Thank You!

Day 1 Complete

See you tomorrow for **Prompt Engineering Deep Dive!**

KavinSchool